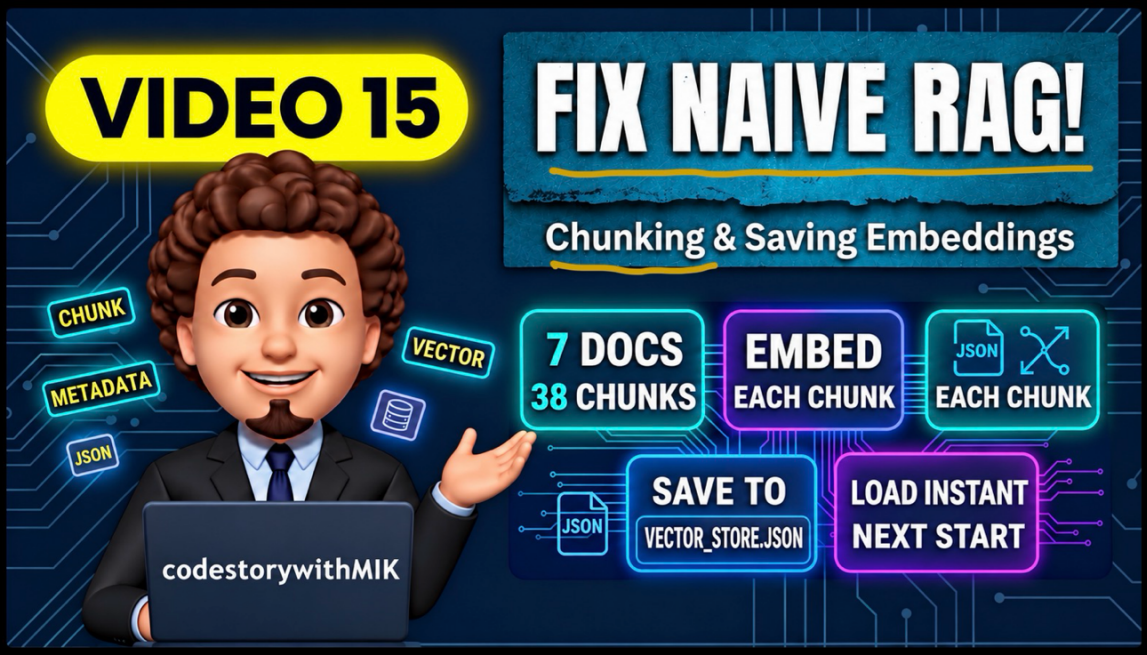


codestorywithmik

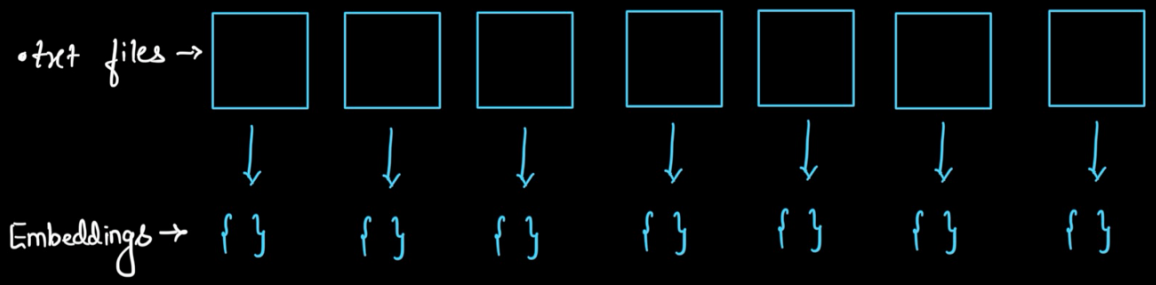
CsWithMIK

codestorywithMIK

Video-14 Recap

Video-14 Recap

Knowledge base (KB) - .txt files



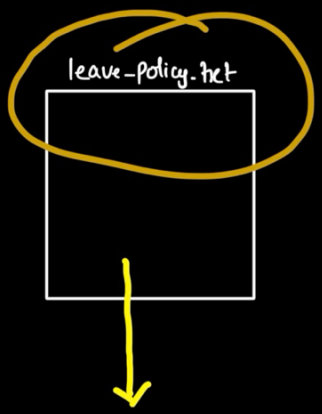
Search the best top-K

Query → "How many days of maternity leave employee gets" → { }

one .txt file contain other informations as well which are not relevant.

I wanted you to feel the need of chunking.

Need for Chunking

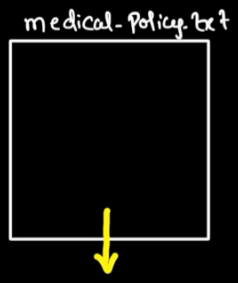


sick leave days, whether hospitalization needs a medical certificate, how long you can be off, and whether it's paid.

User_query

"How many days of leave can I take if I'm hospitalized, and what costs are covered?"

{ }



hospitalization coverage limits, what's reimbursed, and the claim process.

Score

medical_policy.txt → 0.78

leave_policy.txt → 0.74

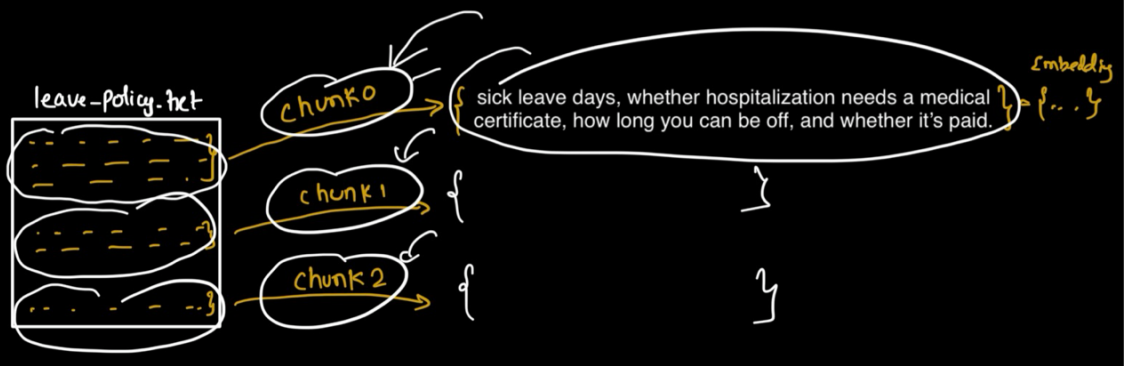
top_K = ~~1~~ 3 → medical_policy.txt **wins**

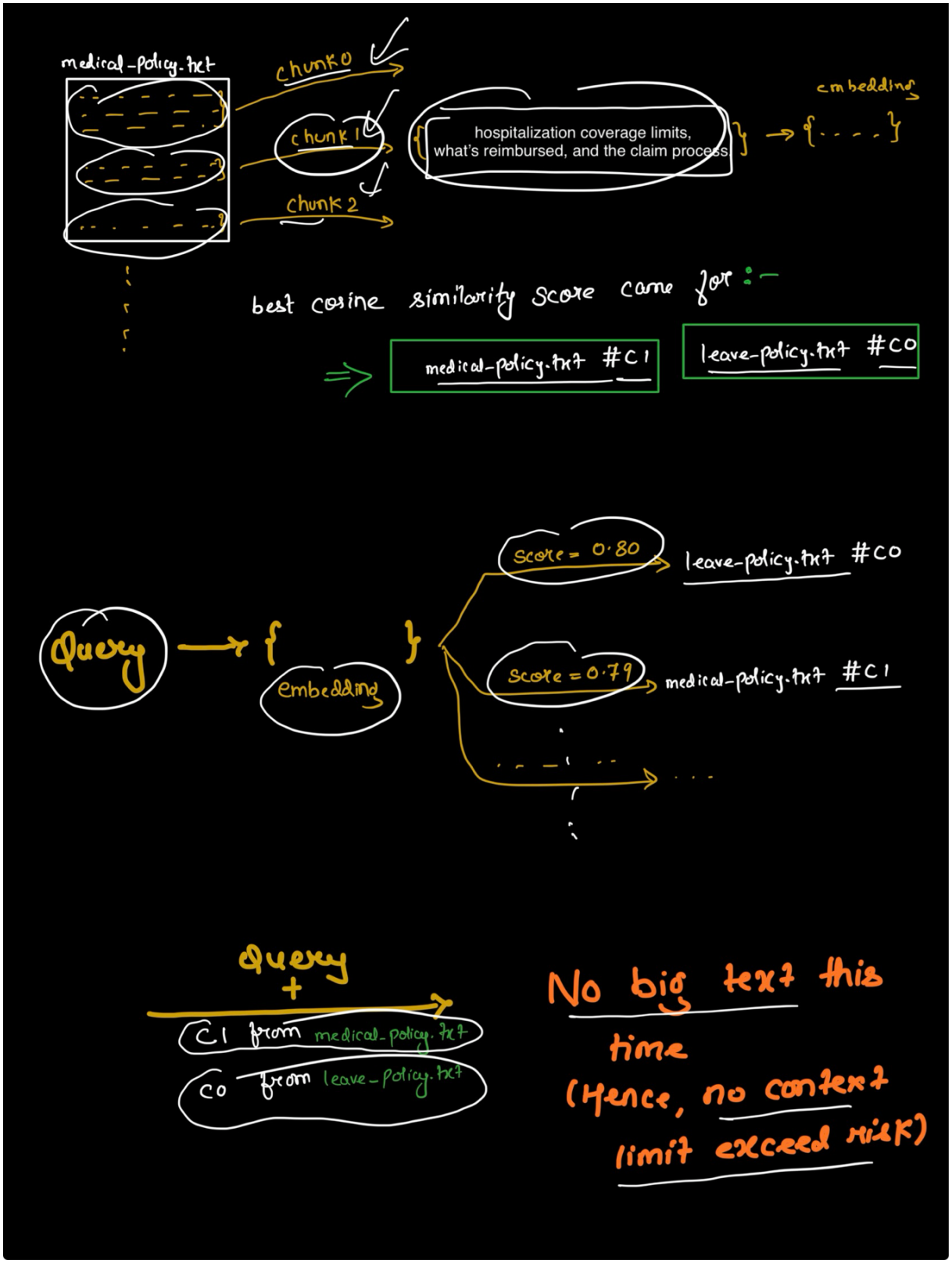
But, Query has two intents → "leave days"
→ "hospital costs"

why can't we give both to LLM ???

Query
+
{ context from medical_policy.txt
{ context from leave_policy.txt
- - — — .

Too big context (limit exceed)




medical_policy.txt
chunk 0
chunk 1
chunk 2
hospitalization coverage limits, what's reimbursed, and the claim process
embedding
{ . . . . }
best cosine similarity score came for :-
medical-policy.txt #C1
leave-policy.txt #C0
Query
{ }
embedding
score = 0.80
leave-policy.txt #C0
score = 0.79
medical-policy.txt #C1
Query
+
C1 from medical_policy.txt
C0 from leave_policy.txt
No big text this time
(Hence, no context limit exceed risk)

Chunk : Small piece of a document, focused enough to match a query and complete enough to be understood on its own.

Full file as context ✗ (Looses context)

Chunks as context ✓ (Crisp & to the point)

Chunk size ?

→ No standard fixed size

→ But 512 to 1204 tokens (200 to 800 words) is very common.

→ But specific size depends on specific data type and retrieval goals.

type and retrieval goals.

In our testing, we will use

One section = One chunk

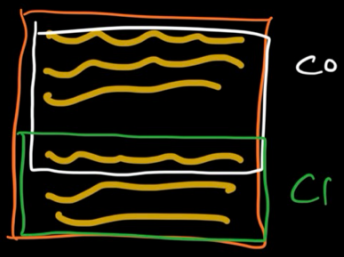
Note: You can use different strategies.

Let's look into the file once.

Note :⇒ Real systems often use recursive splitters that try
- Headings
  - Then, Paragraphs
    - Then, sentences

↳ Then, sentences until a piece (chunk) fits a size limit.

=> Some systems add overlapped strings as well in chunks.



Let's Code

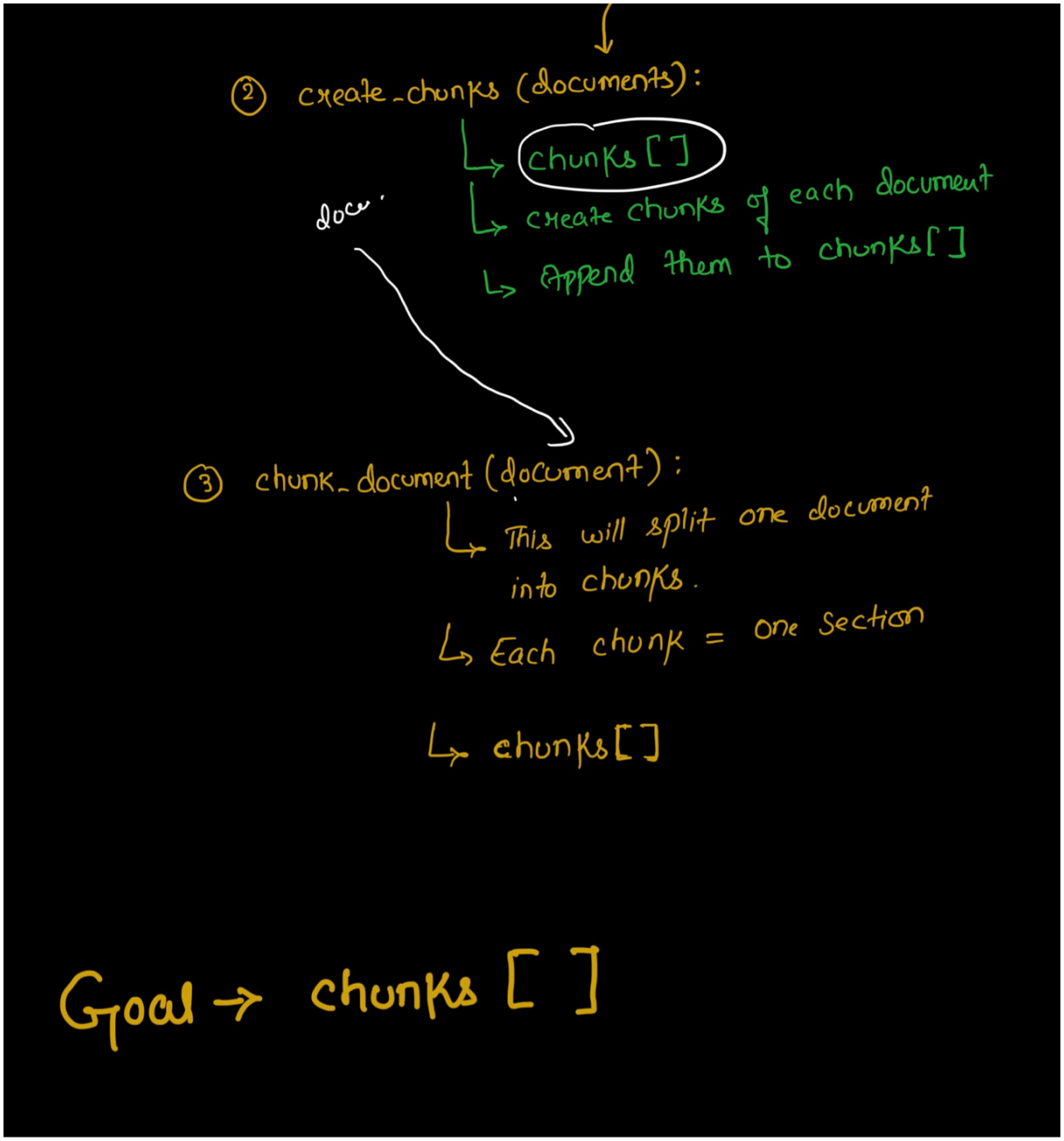
Chunking our docs ...

File :

Code Snippets / chunk_search.py

Video-14 (1) load_documents (KB_FOLDER):

↳ documents [ ] →


② create_chunks (documents):
chunks [ ]
create chunks of each document
Append them to chunks [ ]
docu
③ chunk_document (document):
This will split one document into chunks.
Each chunk = one section
chunks [ ]
Goal → chunks [ ]

Before → one dictionary for whole file.

documents[ ]


```
{
  "source": "leave_policy.txt",
  "text": "Leave Policy - Casual Leave\nEvery employee at Zorvex ... \n\nLeave Policy
}
```


After → chunks of each documents

chunks[ ]


```
chunk 0 = {"source": "leave_policy.txt", "chunk_id": 0,
           "text": "Leave Policy - Casual Leave\nEvery employee at Zorvex Technologies
           "vect":

chunk 3 = {"source": "leave_policy.txt", "chunk_id": 3,
           "text": "Leave Policy - Maternity Leave\nMaternity leave is 26 weeks of pai

chunk 7 = {"source": "leave_policy.txt", "chunk_id": 7,
           "text": "Leave Policy - Applying for Leave\nLeave must be applied for in th
```


Then we will add embedding vector of each chunk in chunks[ ].

An example chunk...

```
{"source": "leave_policy.txt", "chunk_id": 3,
 "text": "Leave Policy - Maternity Leave\nMaternity leave is 26 weeks ...",
 "vector": [0.0123, -0.0456, 0.0789, ... 768 numbers in total]}
```

Note: you can put more info if you want.

Let's code it up!!!

There is One more Problem...

STEP: Knowledge Base → chunking → Embeddings

If we run the program, the

KB → chunking → Embeddings

step is done again.

- It's a time taking step
- KB can be huge

Why don't we save chunks in a storage and next time we load it from there...

For our AI agent, we will store chunks = [ ]

chunks → Local storage (In a file)

```
[
  {
    "source": "hiring_process.txt",
    "chunk_id": 0,
    "text": "Hiring Process - Interview Stages\n...",
    "vector": [0.01, -0.04, ...]
  },
  {
    "source": "hiring_process.txt",
    "chunk_id": 1,
    "text": "Hiring Process - Offer\n...",
    "vector": [ ... ]
  },
  ...
]
```

vector_store.json

Now, next time, when we start, before chunking, we check if we have vector_store.json present or not.

If present, we load it to our chunks[ ]

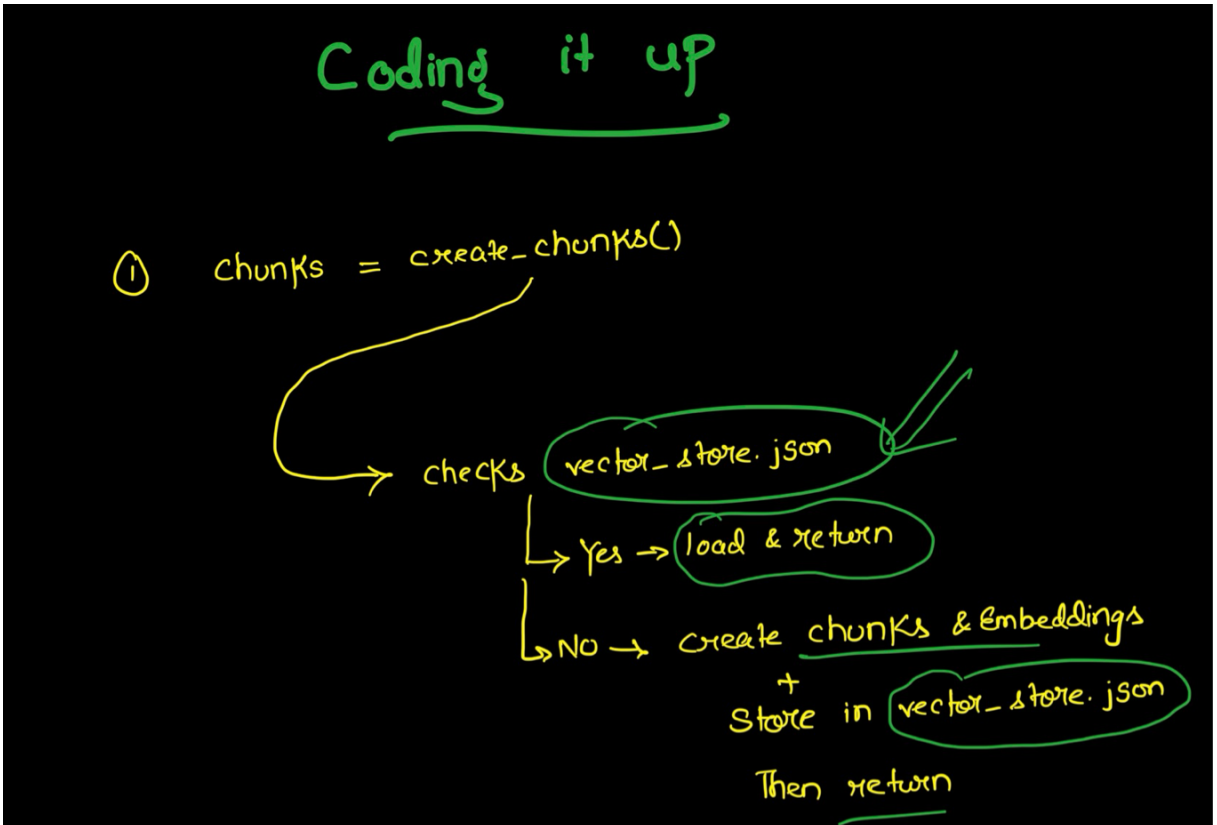
Sample

chunks=[]

knowledge_base > {} vector_store.json > { } 1 > [ ] vector > # 4


```
[
    {
        "source": "hiring_process.txt",
        "chunk_id": 0,
        "text": "Hiring Process - Interview Stages\nCandidates go through 5 stages: resume screening, technical round 1, technical ro
        "vector": [-0.038012005388736725, -0.024958575144410133, -0.17570635676383972,0.030467942357063293,0.030467942357063293],
    },
    {
        "source": "hiring_process.txt",
        "chunk_id": 1,
        "text": "Hiring Process - Offer\nThe offer letter is sent within 5 working days after the final round. A candidate has 7 days
        "vector": [-0.03804845738336725, -0.347394574410133, -0.38574856383972,0.425454545063293,0.536563334463293]
    },
    {
        "source": "hiring_process.txt",
        "chunk_id": 2,
        "text": "Hiring Process - Referral Bonus\nEmployees who refer a candidate that joins get a referral bonus of 25,000 rupees, p
```

Coding it up



2 we can store file name in our .env file.

.env

```
BASE_URL=http://127.0.0.1:11434/v1
API_KEY=dummy_key
MODEL=qwen3:4b
DEBUG=False
EMBED_MODEL=nomic-embed-text
KB_FOLDER=knowledge_base

#Video-15
VECTOR_STORE_FILE=knowledge_base/vector_store.json
```

How to load vector_store.json to chunks[] ???

```
def load_vector_store():
    with open(VECTOR_STORE_FILE, "r", encoding="utf-8") as f:
        return json.load(f)
```

```
chunks = load_vector_store()
```

How to save chunks to vector_store.json file ???

```
def save_vector_store():
    with open(VECTOR_STORE_FILE, "w", encoding="utf-8") as f:
        json.dump(chunks, f, indent=4)
```

Some limitations

- The saved vector_store.json file can become stale/outdated if our KB (Knowledge Base) changes.
- KB → chunks → Embeddings → nomic_embed_text
  - if you now change the model,
  - user-query → embedding with a different model
  - → No more valid.

↳ Both are of different models, can't be compared

↳ Can be of different dimensions embedding vector.

Example:- mxbai-embed-large

↳ 1024 dimension embedding vector

Fix: We can store extra information in the dump like

{"embed_model_used": EMBED_MODEL}

And before loading, we can compare current embedding model with stored model name.

```python
#Get the embedding of a text from Embedding model
def get_embedding(text):
    response = client.embeddings.create(
        model=EMBED_MODEL,
        input=text
    )
    return response.data[0].embedding
```

You can pass multiple strings in a list.

Why embed one chunk at a time.

This can be improved easily.

≈ 40 ⇝ 4 times

Benefits:





① Cut down HTTP round trips again & again

② Much faster usually vs one call per chunk

Note:- You can do them in batch of size 5/10/15 etc...

(*) OpenAI allows upto 2048 strings to input.